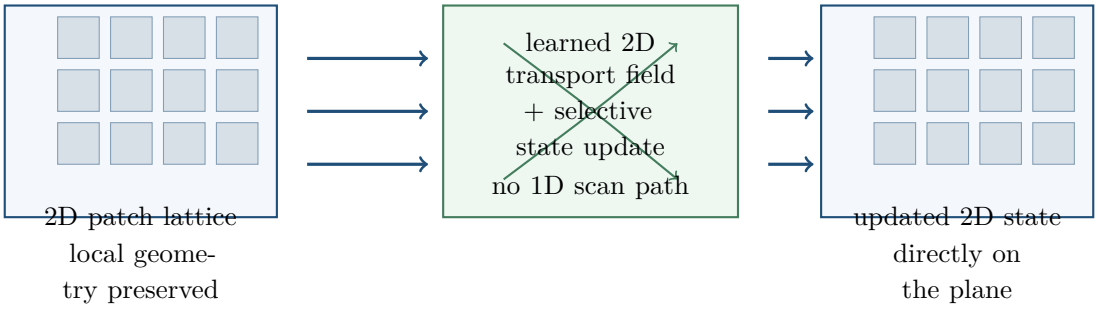
Characteristic Mamba

A 2D Selective Transport State Space Model for Vision

Core idea. Standard Vision Mamba gets long-range transport by forcing an image onto one or more scan paths. Characteristic Mamba keeps the image on its native 2D lattice and performs the recurrence directly on the plane: each patch pulls its next state from an *upstream location* determined by a learned 2D transport field, then applies a selective state update.



What this document gives you. A self-contained methodology section designed so you can implement the model: notation, equations, algorithm, complexity, practical design choices, and an ablation plan. The focus is a strong, defensible mechanism rather than cosmetic architectural changes.

1 Methodology Overview

Let an image be embedded into a patch lattice

$$X \in \mathbb{R}^{B\times H\times W\times D}, \qquad N = HW,$$

where B is the batch size, $H \times W$ is the spatial grid of patches, and D is the channel dimension. The central goal is to preserve the native 2D geometry of the image while retaining the selective state-space flavor that makes Mamba attractive.

The proposed block contains three ingredients:

1. **2D characteristic transport.** Each spatial position predicts where its next state should come from on the image plane. This is a recurrence on the lattice itself, not on a flattened 1D scan path.
2. **Selective state update.** The transported state is filtered by input-dependent retention and injection gates, preserving the selective state-space principle.
3. **Optional local phase rotation.** Paired channels are rotated in a small complex-style plane to keep the useful "oscillatory mixing" intuition from V4 without relying on fragile FFT or exact PDE machinery.

Main modeling choice. A scan is just a recurrence along a fixed path. Characteristic Mamba replaces hand-crafted scan paths with *learned 2D characteristics*. This preserves spatial structure and still gives directed long-range information transport after several recurrent substeps.

2 State Representation

We use a paired state representation inspired by the complex-plane intuition from V4, but implemented with ordinary real tensors for hardware stability:

$$H_t = (U_t, V_t), \qquad U_t, V_t \in \mathbb{R}^{B\times H\times W\times D_s},$$

where D_s is the hidden state width inside the block. The paired channels can be interpreted as two coupled state components. They are produced from the current token features through a linear projection:

$$(U_0, V_0) = \Pi(X).$$

This representation keeps the beneficial "wave-like" coupling idea from V4, but no exact complex arithmetic is required.

3 Learned 2D Characteristic Transport

3.1 Transport field

At each block, predict a 2D transport field on the patch lattice. The simplest version predicts a per-location velocity

$$\mathbf{v}(p) = (v_x(p), v_y(p)) \in \mathbb{R}^2, \qquad p \in \{1,\dots,H\} \times \{1,\dots,W\}.$$

A stronger and more physically structured version predicts a scalar *stream function* ψ, then derives a divergence-free velocity field:

$$\mathbf{v}(p) = \nabla^{\perp}\psi(p) = \Big(\partial_y \psi(p),\ -\partial_x \psi(p)\Big).$$

The stream-function parameterization is recommended because it encourages area-preserving transport and makes the spatial-flow story cleaner. In practice, compute finite differences with fixed kernels or small convolutions.

Recommended implementation. Predict G flow groups instead of a single field. Each group controls D_s/G channels. This gives richer anisotropic transport without quadratic cost. Typical starting values: $G \in \{4, 8\}$.

3.2 From velocity to routing weights

To keep the block TPU-friendly and strictly linear-time, avoid arbitrary all-to-all warping. Instead, convert the predicted velocity into a small set of routing weights over a fixed neighborhood $\mathcal{N}(p)$, for example the 4-neighbor or 8-neighbor stencil.

For the 4-neighbor stencil $\mathcal{N}(p) = \{\text{left}, \text{right}, \text{up}, \text{down}\}$, define direction logits as

$$\ell_{\text{right}}(p) = \kappa v_x(p), \qquad \ell_{\text{left}}(p) = -\kappa v_x(p),$$
$$\ell_{\text{down}}(p) = \kappa v_y(p), \qquad \ell_{\text{up}}(p) = -\kappa v_y(p),$$

where $\kappa > 0$ controls sharpness. The routing weights are

$$\pi_d(p) = \frac{\exp(\ell_d(p))}{\sum_{d' \in \mathcal{N}(p)} \exp(\ell_{d'}(p))}, \qquad d \in \mathcal{N}(p).$$

These weights say how strongly position p should pull information from each neighbor. For each state tensor $S_t \in \{U_t, V_t\}$, define the transported state as

$$\widehat{S}_t(p) = \sum_{d \in \mathcal{N}(p)} \pi_d(p)\, S_t(p + d).$$

This is the discrete 2D analogue of moving information along characteristics, but implemented using only local gather-like operations.

Why this is not just another convolution. The neighborhood is fixed, but the routing weights are *input-dependent and direction-dependent*. A standard convolution applies the same kernel everywhere; here, every location chooses its own transport direction at every forward pass.

4 Selective State Update on the Plane

After transport, each location updates its state using input-dependent selective gates. Let $Z(p)$ denote the current block input feature at position p. Predict three quantities from Z using lightweight pointwise projections:

$$A(p) = \sigma(W_A Z(p)), \qquad B(p) = \sigma(W_B Z(p)), \qquad \Theta(p) = W_\theta Z(p).$$

Here:

- $A(p)$ is the retention gate,
- $B(p)$ is the injection gate,
- $\Theta(p)$ is an optional phase angle used for channel-pair rotation.

4.1 Optional local phase rotation

To retain the most useful part of the V4 complex-plane intuition, apply a per-location channel-wise rotation after transport:

$$\overline{U}_t = \cos(\Theta) \odot \widehat{U}_t - \sin(\Theta) \odot \widehat{V}_t,$$
$$\overline{V}_t = \sin(\Theta) \odot \widehat{U}_t + \cos(\Theta) \odot \widehat{V}_t.$$

This preserves the "oscillatory mixing" flavor without claiming exact Schrödinger evolution.

4.2 Selective recurrence

Let $(X_U, X_V) = \Gamma(Z)$ be the input injection features. The update is

$$U_{t+1} = A \odot \overline{U}_t + B \odot X_U,$$
$$V_{t+1} = A \odot \overline{V}_t + B \odot X_V.$$

This is the key Mamba-style ingredient: the state is not recomputed from scratch; it is *selectively propagated* and *selectively injected* based on the current input.

Important claim discipline. This block is *not* exact unitary quantum evolution. The right story is: **2D selective characteristic transport with optional phase-like channel coupling**. That wording is strong, honest, and implementation-aligned.

5 Full Block

A complete Characteristic Mamba block can be written as follows.

5.1 Local preprocessing

First apply normalization and a lightweight local mixer:

$$\widetilde{X} = \mathrm{SiLU}(\mathrm{DWConv}_{3\times 3}(\mathrm{InProj}(\mathrm{LN}(X)))).$$

This stage is cheap and gives the transport module good local features to condition on.

5.2 Recurrent transport loop

Run the characteristic transport recurrence for T substeps:

$$(U_0, V_0) = \Pi(\widetilde{X}),$$
$$(U_{t+1}, V_{t+1}) = \mathcal{F}_{\mathrm{char}}(U_t, V_t, \widetilde{X}), \qquad t = 0, \ldots, T-1.$$

5.3 Projection back to token space

After T substeps, map the paired state back to the model width:

$$Y = W_o[U_T \| V_T] + D \odot X,$$

where D is a learned skip scale. The block output can then be passed to the next stage exactly like a standard Mamba-style backbone block.

6 Algorithm

Algorithm 1: Characteristic Mamba block

1. Input tokens $X \in \mathbb{R}^{B \times H \times W \times D}$.
2. Normalize and locally preprocess to obtain $\widetilde{X}$.
3. Initialize paired state $(U_0, V_0) = \Pi(\widetilde{X})$.
4. For $t = 0, \ldots, T-1$:
   - 4.1. Predict stream function ψ or velocity field $\mathbf{v}$ from $\widetilde{X}$.
   - 4.2. Convert $\mathbf{v}$ into neighborhood routing weights π_d.
   - 4.3. Transport the state: $(\widehat{U}_t, \widehat{V}_t)$.
   - 4.4. Optionally rotate the paired channels by Θ.
   - 4.5. Predict selective gates A, B and injection features (X_U, X_V).
   - 4.6. Update (U_{t+1}, V_{t+1}) using the selective recurrence.
5. Concatenate (U_T, V_T), project back, and add the residual skip.

7 Complexity Analysis

Let $N = HW$ be the number of patches, D_s the state width, G the number of flow groups, and $|\mathcal{N}|$ the neighborhood size (4 or 8). Then:

- transport routing costs $O(NG)$ to predict and normalize,
- state transport costs $O(ND_s|\mathcal{N}|)$,
- selective gating and projection costs $O(ND_s)$.

Hence the recurrent transport loop is

$$O(T \cdot N \cdot D_s \cdot |\mathcal{N}|),$$

which is linear in the number of patches for fixed T and fixed neighborhood size. This preserves the fundamental efficiency advantage you want over quadratic token-token attention.

Why this is a stronger linear-time story than scans. Multi-scan Vision Mamba gets long-range transport by forcing the recurrence to follow several hand-designed routes. Characteristic Mamba keeps recurrence on the image plane itself and learns the transport direction per location. The cost remains linear because each patch communicates through a constant-size local stencil at each substep.

8 Implementation Blueprint

8.1 Recommended default design

Component	Recommendation	Why
State width D_s	Match the block expansion width or use $D_s = D$	Keeps projection simple early on
Flow groups G	4 or 8	Adds directional richness without large cost
Neighborhood	4-neighbor first, then try 8-neighbor	Easier and more TPU-friendly
Substeps T	2 to 4	Enough to test directed long-range transport
Field parameterization	Stream function ψ if possible	Gives a more structured flow than arbitrary vectors
Rotation	Keep it on for the main model, ablate it later	Retains the V4 intuition without fragile numerics
Gates	Sigmoid retention and injection gates	Closest to selective state-space logic

8.2 Practical notes

Efficient TPU implementation.

- Prefer fixed-size neighbor routing over bilinear warping for the first version.
- Implement neighbor gathering with padding + shifted tensors or grouped depthwise operations.
- Keep all tensors real-valued; no FFT and no complex dtypes are required.
- Predict transport, phase, and gates from the same local conditioning tensor $\widetilde{X}$ to avoid redundant compute.

8.3 Pseudo-code skeleton

```
def char_mamba_block(x):
    z = local_preprocess(norm(x))
    u, v = state_proj(z)
    for t in range(T):
        psi = stream_head(z)          # or a direct velocity head
        vx, vy = stream_to_velocity(psi)
        pi = velocity_to_routing(vx, vy)
        u_hat = transport(u, pi)
        v_hat = transport(v, pi)
        theta = phase_head(z)
        u_rot = cos(theta) * u_hat - sin(theta) * v_hat
        v_rot = sin(theta) * u_hat + cos(theta) * v_hat
        a = sigmoid(a_head(z))
        b = sigmoid(b_head(z))
        xu, xv = inj_proj(z)
        u = a * u_rot + b * xu
        v = a * v_rot + b * xv
    y = out_proj(cat([u, v])) + skip_scale * x
    return y
```

9 Why This Should Improve Over V4

V4 already showed that preserving 2D structure can work, but it appears to plateau because the local spatial operator is too isotropic and too uniform. Characteristic Mamba changes the communication law itself:

- **Directed transport instead of isotropic mixing.** Hard classes often require information to travel along object-specific directions.
- **Spatial recurrence on the plane.** The state moves where the image says it should move, not along a hand-crafted scan order.
- **Selectable memory.** The retention/injection gates keep the Mamba-style selectivity rather than turning the block into a pure spatial smoother.
- **Complexity discipline.** The model gains long-range transport after several recurrent substeps while keeping constant-size local communication at each step.

10 Ablation Plan You Should Run

Ablation	Question it answers
V4 vs Characteristic Mamba	Does learned 2D transport break the plateau?
Fixed velocity vs learned velocity	Is adaptivity of the transport field necessary?
Arbitrary velocity vs stream-function velocity	Does structured, divergence-free transport help?
With vs without phase rotation	Does the paired-state rotation matter beyond transport alone?
4-neighbor vs 8-neighbor routing	How much local routing richness is needed?
1, 2, 4 transport substeps	Are gains coming from longer recurrent transport depth?
1 flow group vs 4 or 8 groups	Does channel-group-specific transport improve expressivity?

11 Final Positioning for the Paper

The strongest methodology story is not "we solved vision with a PDE" and not "we replaced attention with another token mixer." The strongest story is:

Characteristic Mamba performs selective state-space recurrence directly on the image plane. It keeps the geometry of the 2D lattice, learns transport directions instead of using fixed scan routes, and preserves linear-time complexity by restricting each update to a constant-size neighborhood.

That framing is tight, implementation-aligned, and easy for a reviewer to understand.

Minimal version to build first. Start with: one stream-function field, 4-neighbor routing, 2 paired state tensors, 2–4 recurrent transport substeps, and simple sigmoid gates. Get this stable first. After that, the most meaningful extension is flow groups, not random extra convolutions.